

02 Procedimiento de implementacion SaborExpress

SABOREXPRESS — PROYECTO DE SOFTWARE

Yerson Stiven Cuellar Rubiano

Ficha de formación: 3145555

Expediente técnico y académico — APA 7

14 de octubre de 2026

Fuente documental: repositorios SaborExpressBack, SaborExpressWeb y SaborExpressMovil.

Las pruebas no ejecutadas se identifican como pendientes.

Procedimiento de implementación y liberación

Proyecto: SaborExpress | **Autor / integrante:** Yerson Stiven Cuellar Rubiano | **Ficha:** 3145555 | **Fecha:** 14 de octubre de 2026 | **Versión documental:** 4.0

Estado de la evidencia: inspección documental y estática del código. Las pruebas de ejecución y los datos institucionales no aportados no se presumen realizados.

1. Punto de partida

Existen tres proyectos de aplicación: SaborExpressBack, SaborExpressWeb, SaborExpressMovil. Se recomienda definir el SHA de liberación por repositorio; versiones de paquetes no implican que el despliegue haya ocurrido. Requiere un ambiente de pruebas representativo y un responsable de cambios.

2. Secuencia de implementación (plan repetible)

1. Revisión de requisitos y congelamiento temporal de ramas. 2. Validación de dependencias y secretos. 3. Restauración/compilación API. 4. Aprovisionamiento DB y generación/revisión de migraciones. 5. Ejecución de tests unitarios/integración. 6. Compilación de Next.js. 7. Compilación/ejecución del cliente Expo. 8. Pruebas de contrato Swagger y seguridad. 9. Sandbox Wompi. 10. Ensayos con roles/sedes/usuarios ficticios. 11. Ensayo de respaldo/restauración y rollback. 12. Aprobación formal.

3. Control por etapa

Etapa	Entrada	Validación	Salida
Preparar	Ramas + configuración	Versiones y secretos ocultos	SHA por repo
Construir	Código + SDKs	dotnet build / npm run build	Artefactos
Migrar	DB efímera	Generación SQL e idempotencia	Esquema documentado
Integrar	API/web/móvil	Pruebas E2E/SignalR/Wompi	Registros de QA
Aprobar	Evidencias	Checklist y firmas	Acta de aceptación
Publicar	Infra autorizada	Smoke y monitoreo	Release trazable

4. Reversión

Ante falla, detener promoción, revisar integridad de la base y restaurar copia si el cambio de esquema lo necesita; volver a binarios conocidos, suspender tráfico de pago hasta reconciliar

transacciones pendientes y registrar el incidente. No usar una reversión de Git como reemplazo de un procedimiento de rollback de datos.

5. Documentos de control

Acta de cambios con fecha, versión, ventana, responsables, ticket, evidencias de QA, historial de migraciones, lista de secretos rotados, resultado de smoke y autorización. Ninguna fecha de despliegue ni firma se completa artificialmente.

7. Runbook de implementación

ID	Paso	Acción	Puerta / evidencia
PRE-01	Congelar código	registrar hash `git rev-parse HEAD` para API, Web, Móvil y Docs	revisión de dependencias y responsables
PRE-02	Revisión de secretos	inventariar nombres de variables en configuración y asegurar gestores	no guardar secretos en commit
PRE-03	Respaldo	generar y verificar copia de BD y archivos de prueba	prueba restauración aislada
PRE-04	Compilar y preparar	dotnet restore/build, npm ci/build, npm ci móvil	artefactos versionados por SHA
PRE-05	Migrar esquema	comparar SQL de migración y motor escogido	aplicación aprobada en staging
PRE-06	Habilitar API	ejecutar en entorno con proxy TLS y CORS explícito	Swagger protegido según ambiente
PRE-07	Publicar web	build Next.js y configuración de endpoint API	pruebas auth, catálogo y pedido
PRE-08	Empaquetar móvil	seguir flujo Expo EAS/versiones correctas	smoke Android/iOS aplicables
PRE-09	Validar pasarela	pruebas Wompi sandbox firmadas y duplicados	transacción consistente
PRE-10	Aprobación/reversión	firmar resultados, monitorizar y guardar rollback	responsable habilita release

8. Integración y consistencia entre repositorios

Se recomienda mantener una matriz con commit SHA de los cuatro repositorios, versión funcional de API, variables por entorno, compatibilidad de la app móvil y cambios del esquema. Los cambios de DTO que afectan pedidos/pagos deben anunciarse en el mismo ciclo a web y móvil. El repositorio Docs recibe los diagramas, contratos, guías y resultados de QA asociados a esas revisiones.

Ruta de salida al mercado: develop → staging → release con aprobación de la persona responsable; evitar desplegar directamente cambios no revisados desde la rama de trabajo. La

rama dev citada por el usuario corresponde a documentación; no presupone que el backend utilice la misma rama.

9. Validaciones post-despliegue y reversión

Smoke por ambiente: verificar que API responde sin error; consultar catálogo con y sin branchId; inicio de sesión sintético; orden sintética con precios coherentes; intento de acceso a recurso ajeno; Wompi sandbox con evento firmado; tablero cocina con reconexión; lectura de entrega propia.

Monitoreo posterior: errores 5xx, tiempo de inicialización, fallos de migración, pagos pendientes que exceden política acordada, tasa de conexiones SignalR, reclamos y datos fuera de sede. Detener o revertir despliegue si aparecen discrepancias financieras, escalamiento de privilegios o corrupción.

Rollback seguro: 1) retirar tráfico o mantener modo lectura; 2) detener writers; 3) conservar logs; 4) revisar qué migraciones fueron aplicadas; 5) reponer build anterior; 6) restaurar backup únicamente si la compatibilidad de esquema lo exige y tras evaluar pérdida de datos; 7) validar conciliación de pagos externos; 8) registrar incidente, decisiones y próxima ventana. No aplicar DROP DATABASE ni un script genérico de reversión sobre producción.

10. Plan de preparación de ambientes

El entorno local es apropiado para implementar y depurar, pero no permite concluir disponibilidad productiva. Se propone definir tres ambientes claramente separados: desarrollo con datos sintéticos, integración/staging con configuración parecida a producción y producción con secretos y datos reales. Cada ambiente debe tener su propia base de datos, pasarela de pagos y políticas de correo; se prohíbe usar credenciales de producción para pruebas repetibles.

Inventario mínimo por ambiente: identificador del entorno, repositorio y SHA de backend/web/móvil, .NET Runtime y SDK, Node, versión de Expo SDK, proveedor y versión de base de datos, versión de migración EF, origen permitido CORS, URL pública API, URLs de

devolución/webhook y método de almacenamiento seguro de secretos. Documentar fecha de cambio y responsable de cada variable sin almacenar su valor privado en el repositorio.

11. Control de migraciones y persistencia

El código de Program.cs incluye Database.MigrateAsync() antes de exponer el servicio. Este mecanismo simplifica arranque local, pero en producción se recomienda una etapa de migración controlada y única para evitar escrituras simultáneas y cambios no aprobados. Antes de cualquier actualización de esquema se requiere respaldo verificable, revisión del SQL generado y prueba de restauración en una base aislada.

Reglas de ejecución: evitar EnsureCreated() en esquemas gestionados con migraciones; seleccionar proveedor consistente con el ambiente; comprobar tipos decimal, precisiones y conversiones DateTime; validar índices de consultas por pedido, sede, empleado y referencia de pago. Conservar el resultado de la migración y compararlo con versión de aplicación. No publicar database-schema.sql ficticio como prueba de un esquema ejecutado.

12. Liberación de backend

1. Obtener HEAD exacto y comprobar ausencia de cambios locales inadvertidos. 2. Validar restauración y compilación dotnet restore, dotnet build -c Release. 3. Ejecutar pruebas automatizadas disponibles, o registrar su ausencia si no existen. 4. Validar configuración de JWT, proveedor DB, CORS, URL Wompi y política de turno. 5. Ejecutar migración solo en ambiente aprobado. 6. Iniciar servicio y observar errores del arranque. 7. Consultar Swagger autorizado y tres endpoints no destructivos. 8. Confirmar autenticación y aislamiento de sedes con cuentas de prueba. 9. Registrar logs anonimizados, estado de build y aprobación.

13. Liberación web y móvil

Web: instalar versiones del package-lock.json con npm ci, ejecutar npm run lint y npm run build, verificar que las variables públicas no transporten secretos y probar navegación en rutas

/dashboard, /orders, /products, /branches y /reports con cada rol previsto. La compilación no valida autorización de objetos.

Móvil: usar npm ci, comprobar compatibilidad entre SDK Expo, runtime nativo y dispositivo/emulador, generar build de desarrollo si Expo Go no contiene módulos requeridos, apuntar a la API del ambiente correcto y ensayar reconexión, localización, permisos de notificaciones y visualización de órdenes. Evitar publicar una versión a tienda sin política de compatibilidad de API.

14. Procedimiento operativo detallado de comprobación

Etapa	Acción / entrada	Evidencia obligatoria	Si falla
Identidad	Login y renovación de sesión con usuario sintético	HTTP, rol y expiración	Revisar JWT/horario
Catálogo	GET Products con sedes A/B	Productos globales y específicos	Revisar BranchId
Pedido	Crear orden de mesa o domicilio	Total y OrderDetails coherentes	Revisar DTO, validaciones y transacción
Cocina	Cambiar estado permitido	Historial consistente	Revisar política ActiveShift
Pago	Init Wompi sandbox y webhook	Referencia, checksum, estado final	No forzar confirmación manual
Reparto	Asignar empleado y cerrar entrega	Sede/cliente correctos	Investigar acceso y disponibilidad
Recuperación	Caída temporal de red	Reconciliación tras reconectar	Activar fallback a consulta
Observabilidad	Revisar métricas y errores	Acta posdespliegue	Rollback/contención

15. Modelo de acta de implantación

El acta debe registrar: versión/fecha, responsables, entorno, commits de repositorios, lista de migraciones, respaldo asociado, checksums de paquetes, criterios aprobados, casos de prueba con resultado observado y anomalías pendientes. El aprobador funcional debe ser distinto del desarrollador de los módulos críticos cuando sea posible. No convertir casillas no ejecutadas en “aprobado”.

Cierre: registrar versión estable, puntos de recuperación, manual de operación disponible, responsables de guardia, enlace de incidencias y problemas conocidos. Una puesta en marcha de pruebas no equivale a una aceptación de producción.

16. Mantenimiento evolutivo

Cada cambio que modifique una tabla, DTO, permiso, rol o flujo financiero debe abrir un análisis de impacto, un ajuste de la SRS y un caso de regresión; se actualizan API, clientes y documentación como parte del mismo cambio. Para la operación semanal propuesta, priorizar revisión de errores, pagos pendientes, dependencias vulnerables, copias de seguridad y divergencias de catálogo. La periodicidad es una recomendación y puede variar según tráfico y presupuesto.

Fuentes primarias de implementación

- API: https://github.com/yersonct/SaborExpressBack/tree/main
- Web: https://github.com/yersonct/SaborExpressWeb/tree/main
- Móvil: https://github.com/yersonct/SaborExpressMovil/tree/main
- Referencia de presentación: https://github.com/jsgh1/SRD-v1/tree/dev/docs/docs-SRD

Control documental: documento elaborado a partir del estado observado en octubre de 2026. Antes de entregar, verificar el commit exacto, responsable, revisión del instructor y cualquier cambio posterior del código.

14. Arquitectura y orden de arranque

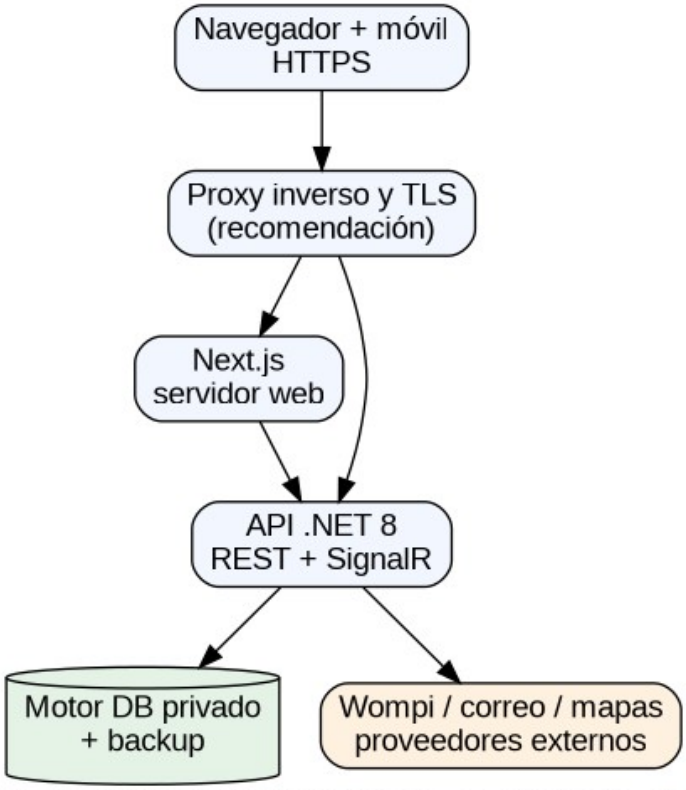


Figura 7. Despliegue lógico PROPUSTO: no es evidencia de hosting actual

07_DESPLIEGUE_PROPUESTO

Nota. Elaboración para el estudio de caso SaborExpress; las figuras no certifican operaciones desplegadas. Primero debe existir un motor relacional de prueba configurado, luego ejecutarse migraciones controladas y después iniciarse la API. Web y móvil necesitan una URL API alcanzable desde su propio entorno; un teléfono no puede usar localhost para referirse al servidor de la computadora. Para pruebas de pagos, Wompi utiliza credenciales y URLs sandbox separadas de producción.

15. Comandos técnicos versionables y seguros

```
# Ejecución en una terminal PowerShell con herramientas instaladas.
git clone https://github.com/yersonct/SaborExpressBack.git
cd SaborExpressBack
```

```
git rev-parse HEAD
dotnet restore
dotnet build -c Release
# dotnet run  # Ejecutar solo tras configurar DB, JWT y servicios externos.
cd ..
git clone https://github.com/yersonct/SaborExpressWeb.git
cd SaborExpressWeb
npm ci
npm run build
# npm run dev
cd ..
git clone https://github.com/yersonct/SaborExpressMovil.git
cd SaborExpressMovil
npm ci
# npm start
```

Criterio: el ejemplo es una guía para reproducir, no una transcripción de comandos ejecutados con éxito. dotnet ef migrations script puede generar DDL, pero debe ejecutarse con proveedor correcto y revisar el resultado antes de publicarlo.